IAT355 Final Project Report

Qian ZhongJun 301541827

Aden Trathen 301451426

User Manual

Live View Setup:

View this notebook in your browser by running a web server in this folder. For example:

~~~sh

npx http-server

~~~

Or, use the [Observable Runtime](https://github.com/observablehq/runtime) to import this module directly into your application. To npm install:

~~~sh

npm install @observablehq/runtime@5

npm install https://api.observablehq.com/d/56ba25a95f74523d@661.tgz?v=3

~~~

Then, import your notebook and the runtime as:

~~~js
~~~

import {Runtime, Inspector} from "@observablehq/runtime";

import define from "56ba25a95f74523d";

~~~

The visualization requires multiple kali Linux vm backend, port forwarded through ssh on port 8080 as per the serve_json.py, use cmd command ssh -L 808x:localhost:8080 kali@192.168.x.x(your kali local ip). (where x is the vm number)

On the blue team kali VM we used tshark -i to serve the data live, 2 sources: TCP && UDP are captured and combined by the tool and served to python, it is important to note to run tshark capture first instead of the python3 server.

network config:

IP assignment: Manual

IPv4 address: 192.168.1.x

IPv4 mask: 255.255.255.0

IPv4 gateway: 192.168.1.1

DNS server assignment: Manual

IPv4 servers: 1.1.1.1

(SSH command) ssh -L 808x:localhost:8080 kali@192.168.1.x

Purpose:

Creates an SSH tunnel that securely forwards traffic from your local machine's port 808x to the remote Kali machine's port 8080.

ssh: Secure Shell — used to remotely access another system securely.
~~~

-L 808x:localhost:8080:

808x: A local port on your machine (e.g., 8081, 8082, etc.).

localhost: Tells SSH to forward to the loopback interface of the remote machine.

8080: The destination port on the remote machine — commonly used by dashboards or Flask servers.

kali@192.168.x.x: SSH user kali on a device with IP 192.168.x.x (e.g., your VLAN 10 Kali laptop).

(tshark command): sudo tshark -i eth0.10 -T fields -e frame.time_epoch -e ip.src -e ip.dst -e ip.proto -E header=y -E separator=, > vlan10_output.csv

sudo: Runs the command with root privileges (required to access interfaces).

tshark: CLI version of Wireshark used for capturing and analyzing network packets.

-i eth0.10: Captures traffic from the VLAN 10 virtual interface.

-T fields: Outputs specific packet fields rather than full packet detail.

-e frame.time_epoch: Captures the packet timestamp in Unix epoch format.

-e ip.src: Captures the source IP address.

-e ip.dst: Captures the destination IP address.

-e ip.proto: Captures the IP protocol number (e.g., TCP = 6, UDP = 17).

-E header=y: Includes column headers in the output.

-E separator=,: Uses a comma as the delimiter (CSV format).

> vlan10_output.csv: Redirects the output to a file for later analysis.

(nmap command): sudo nmap -sV -T4 192.168.1.x

Performs a service/version detection scan on a target host.

sudo: Root access is required to send raw packets.

nmap: A powerful network scanning tool.

-sV: Enables service version detection to determine what software (and version) is running on open ports.

-T4: Uses aggressive timing for faster scanning (less stealthy).

192.168.1.x: Replace x with the actual host's last octet — this is the IP of the target device.

(hping3 command:) sudo hping3 -c 5000 192.168.1.x

Sends 5,000 packets to a target — typically used for traffic simulation, stress testing, or DoS behavior analysis in a lab environment.

sudo: Required for raw packet crafting.

hping3: A packet generator and analyzer (similar to ping but far more customizable).

-c 5000: Sends exactly 5,000 packets to the target.

192.168.1.x: Target IP address — should be replaced with a real host.

To check for ip configuration:

Windows: ipconfig

kali Linux: ip a

To check connectivity: ping 192.168.1.x

It is important to note that both machines need to have appropriate firewall rules set up or disable it altogether to ensure communication.

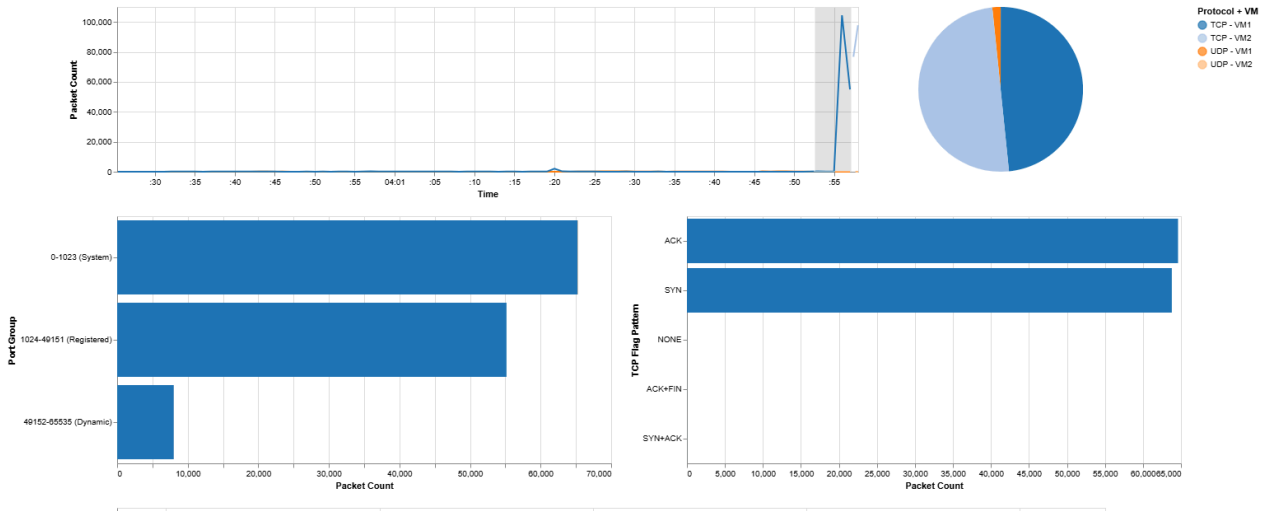
on red team kali VM we plan on using nmap a scan source, hping3 to simulate DoS sources

The physical setup is with 2 windows laptops hosting kali VM to ensure isolation of LAN and a TL SG105E managed switch with VLAN to ensure L2 separation for even more security

Observable NoteBook (Static Visualization):

For the observable notebook, the interactions are very straightforward. The Dashboard is made up of two major parts, the packet count tracker and the packet drill down.

The packet count tracker shows the total amount of packets of each type from each virtual machine on a line graph, allowing the user to see any spikes in traffic that could be related to a DDOS attack or other forms of cyber crime. This section can be brushed, filtering the packet counts within that selected time in the pie chart and bar graphs.



The packet drill down is a way to look more closely at suspicious packets, allowing users to see which IP addresses are sending the most packets. The user can hover over them to bring up a tooltip with their packet size, or to view which destination IP they went to. These Packets are also color coded based on whether they were UDP or TCP, so that users can more readily find the types of packets they are interested in.

Feasibility Pilot

For our Feasibility pilot, we had two of our friends who were familiar but not experts with cyber security, May and Jonathan. They viewed the Observable notebook version of our project over discord. We had them do the

Tasks:

Brush the line graph so that the lowest spike is selected.

View the total amount of Dynamic activity during that time, which parts of it were TCP? UDP? From Virtual machine 1? Or 2?

Brush the line graph so that the highest spike is selected.

View the total amount of System activity during that time, which parts of it were TCP? UDP? From Virtual machine 1? Or 2?

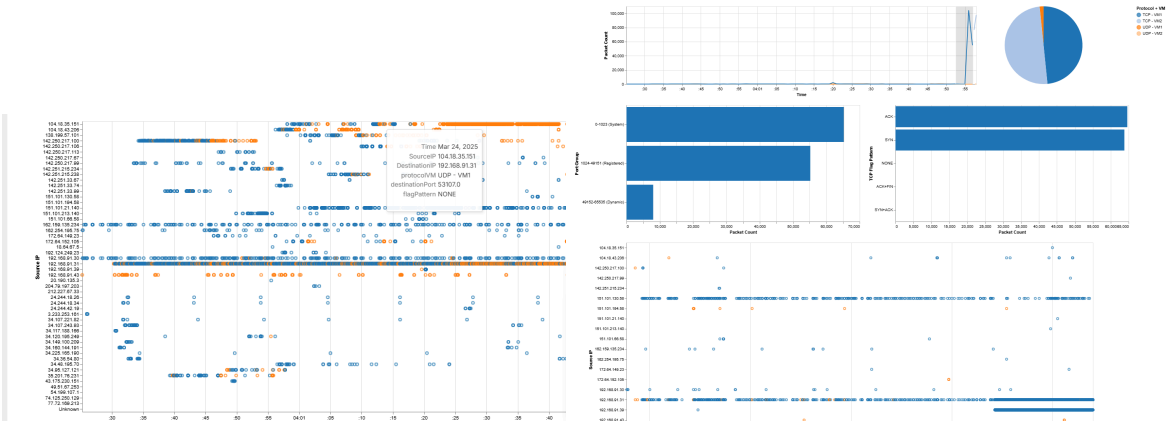
View the packet drill down, which IP address is responsible for the attack?

View an individual packet from that address, what is its packet size? Where is it going?

When did the attack happen?

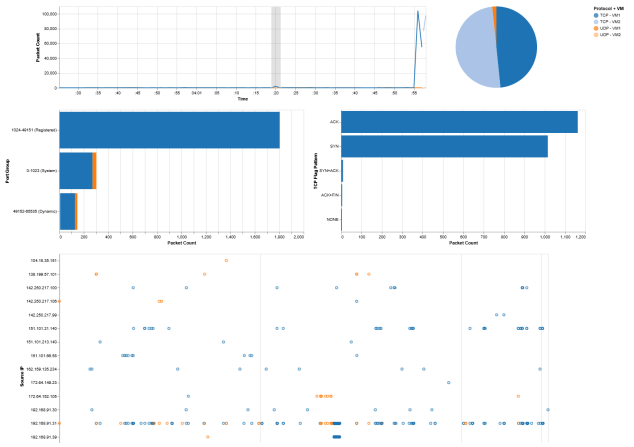
When did the scan happen?

May had no problem completing the tasks, quickly finding the relevant sections and extracting answers from them.



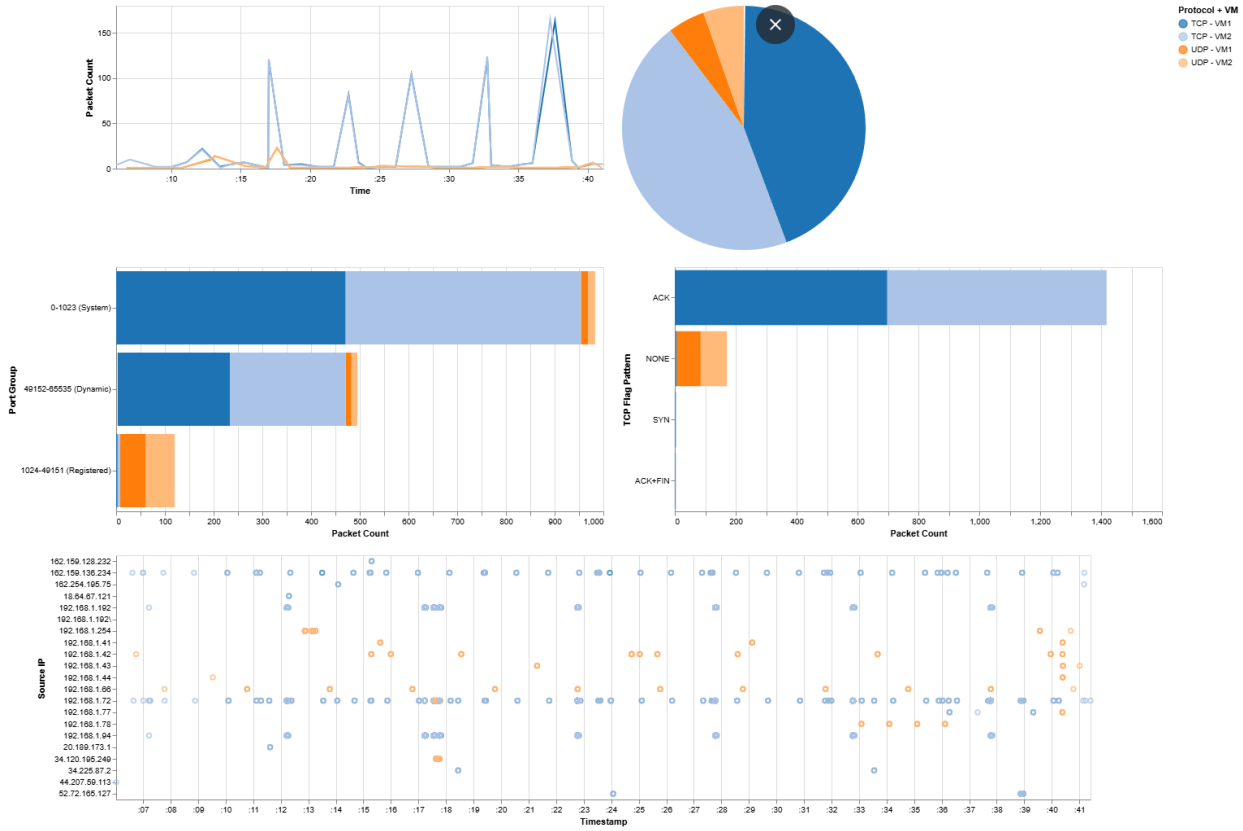
May recommended that we could add a feature to remove certain IP addresses from the packet drill down. This could be a helpful feature, if certain addresses are not a concern, however it was a little outside of the project scope.

Jonathan had some struggles finding things like the dynamic activity port group. This may be something related to how the questions were worded, but may also be an issue with the design if it's hard to find those labels.



Jonathan didn't have any recommendations outside of potentially rearranging things to make it easier to find certain labels, commenting that some of them were too small.

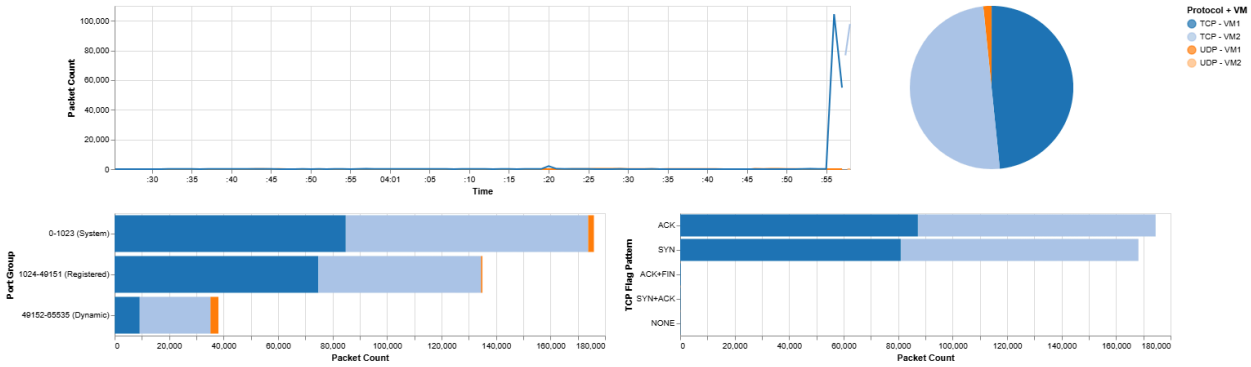
Visualization Overview



The visualization is a network monitoring dashboard, providing users with a large amount of network data in a flash. Our audience is cybersecurity professionals, SOC analysts, penetration testers, and security researchers who require quick and easy interactive visualization tools for network security. In terms of research questions, users will be able to ask, “Is there any unusual packet activity?” “Is there any specific ip address that is putting out a lot of packages? Where are those going?” and find answers to these right away, this could help them identify unprotected areas of their network or respond to cyber attacks more easily.

Our data sets are from the normal background traffic, TCP scans via nmap as well as the SYN floods via hping3 -S, and ICMP floods via hping3 --icmp. Each represents a unique operational pattern, aligned with real-world security event differentiation. In SIEM workflows, such event classes are treated as distinct datasets — not because of how they’re transported, but due to what they signify. While our Observable dashboard receives a unified data stream, the content reflects multiple, semantically distinct data sources.

Monitoring Dashboard:



Data Set Description.

Time (Temporal)(X): Used to show trends in TCP and UDP packet traffic over time.

Protocol Type(Nominal)(Y): Differentiates between TCP and UDP protocols.

Protocol type counts(Quantitative)(PI): Shows which protocol types are more frequent.

Destination Port(Nominal)(Y): Provides insights into which ports are being accessed.

TCP Flag Pattern(Nominal)(Y): Highlights SYN, ACK, FIN, RST patterns in TCP packets.

Packet Count(Quantitative)(X&Y): Displays the total number of packets for each dimension.

Interactions

The dashboard allows users to brush the timeline to view more generalized packet counts, sorted by the TCP flag pattern types. This lets security view spikes right away, as well as determine the port groups that are in use with which packets.

Design Choices

Idiom:

Line chart: Used for showing time-based trends in packet traffic. This helps users see spikes in traffic very clearly.

Bar Chart: Used for displaying categorical distributions, such as ports and TCP flags. This way users will be able to see general counts of this important data quickly.

Pie chart: This chart gives a quick rundown of the number of TCP and UDP Packets from both Virtual Machines.

Encoding:

Identity Channels:

We used colors across the board to make it clear which parts of the visualizations relate to which virtual machine And whether they were UDP or TCP.

Magnitude channels:

Position is used in the line chart to show the spikes in packet counts based on protocol type on the timeline.

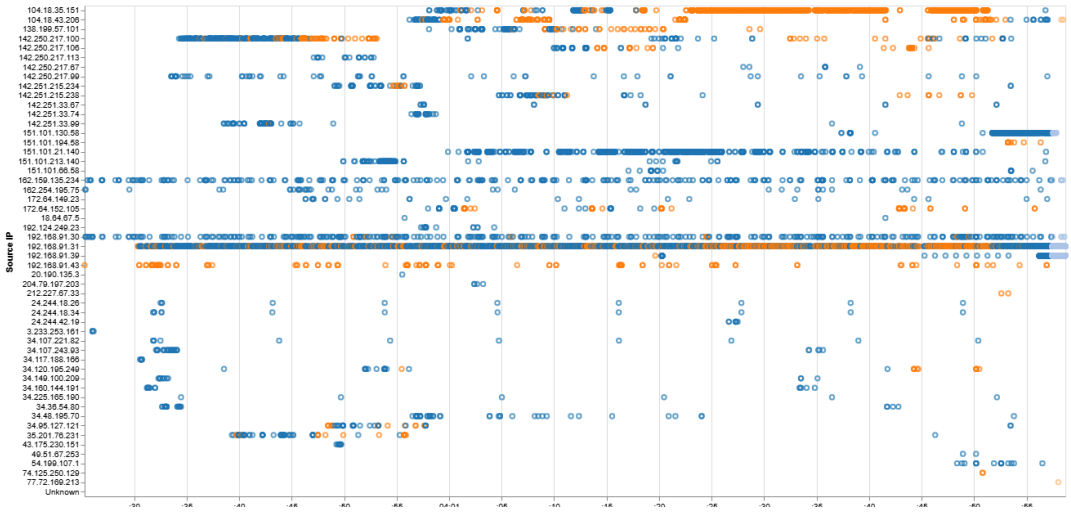
Length is used in the bar charts to help illustrate the amount of packets either by protocol type or port group.

2D area is used in the pie chart to show the ratios between the packets from each machine and each protocol type per machine.

How These Choices Respond to Our Analytical Questions

We wanted to employ human precognitive scanning to our benefit. By creating a splunk type dashboard, cyber security professionals would have an easier type noticing and potentially stopping threats to a network. Aspects like the temporal graph, bar graphs and pie chart make it easy for users to notice irregular activity across the system. We wanted the graphs to be easily readable, so we stuck to basics like the bar, and line graph. While pie charts can be hard for people to glean exact percentages from, we felt like as a generalization of the total data, it was fairly useful.

Packet Drill Down:

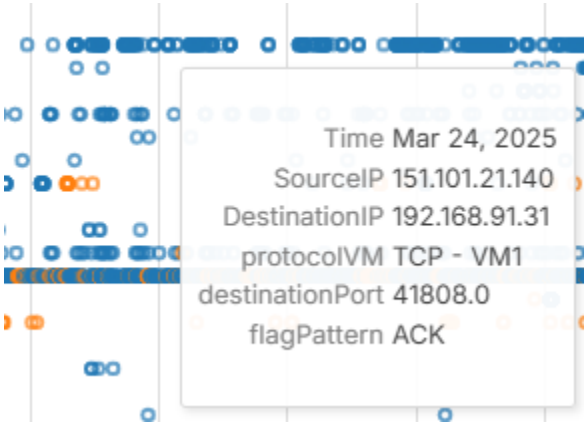


Data Set Description.

Time (Temporal): Used to show trends in TCP and UDP packet traffic over time.

Source IP (Nominal): Allows the security analyst to view a detailed view of all the traffic on the network by their sources.

Interactions



The interaction for the drill down is a simple tooltip, hover over the packets in the right hand graph to see more information about what the packet is. It contains info like the size, type, and sources and destinations.

Design Choices

Idiom:

Scatter Plot: Used for displaying the packets from each port. as it uses the IP addresses as different points on the y-axis of the visualization, it acts a little differently from the norm.

Encoding:

Identity Channels:

We used colors across the board to make it clear which parts of the visualizations relate to which virtual machine And whether they were UDP or TCP.

Magnitude channels:

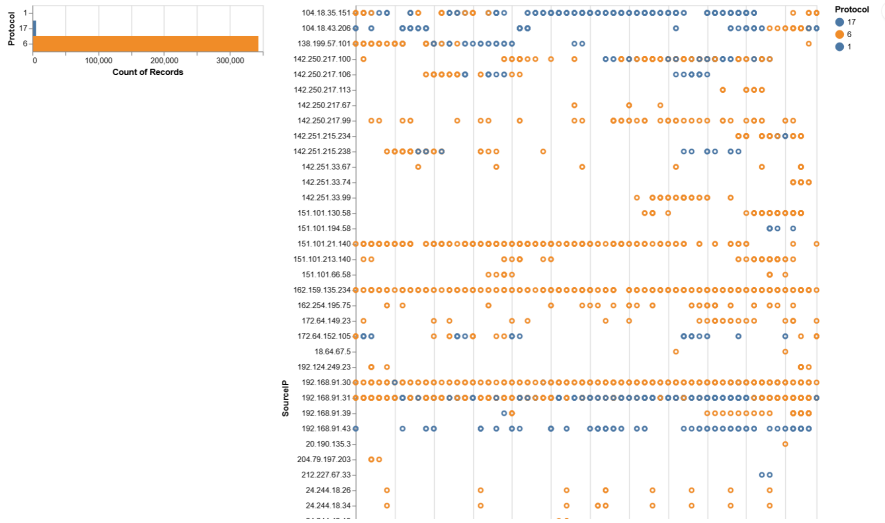
Position on a common scale was used to show the timeline of the attack, packets are displayed in an easy to follow way from the source IP.

How These Choices Respond to Our Analytical Questions

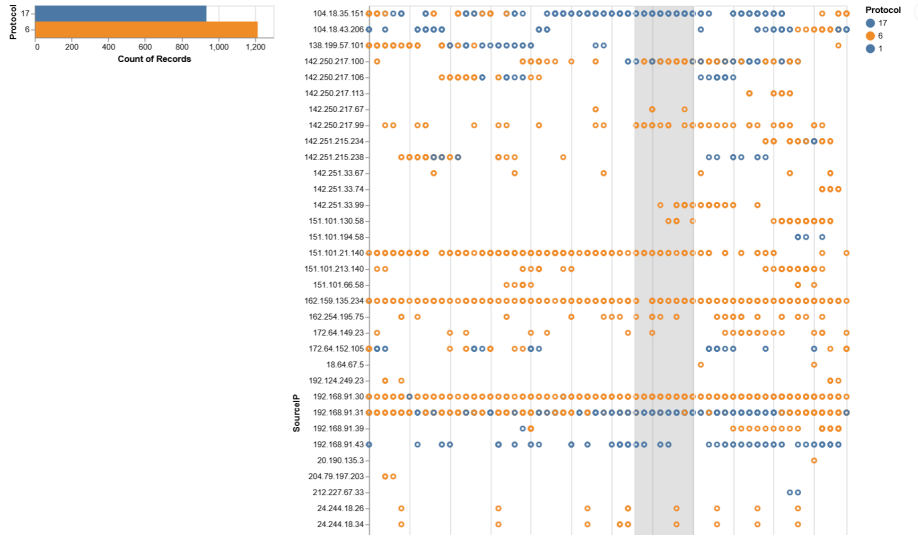
The Packet Drill Down is a useful tool for finding specifics from the packets, allowing professionals to identify which IPs and ports are perpetrating or are victims of attacks.

The timeline creates clear groupings of IP addresses as the packets are added to the visualization based on their timestamp. Then, the tooltip allows them to view other important aspects of the packets like their size and destination IP, allowing for the identification of suspicious packets.

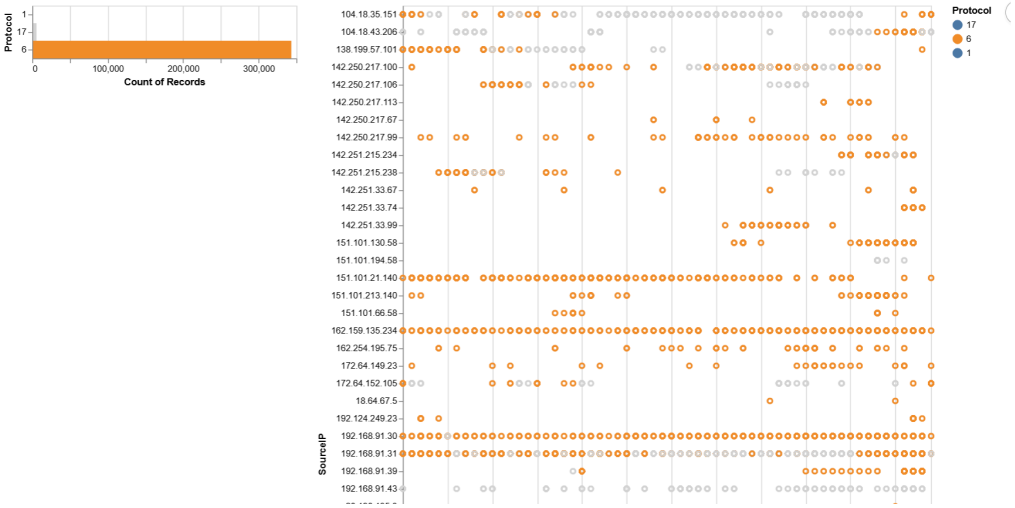
Unimplemented Sections of the Design



TCP and UDP filtering: brushing the packet drill down would provide the user with updated counts of the UDP and TCP count. This would have been useful for getting packet totals from a specific time stamp.



It also could have been done in reverse, greying out the packets that were not selected by the bars in the count. Allowing users to see one protocol they deemed more important than the other.



This started to not work once we tried to implement it as part of the dashboard. It would either lose its functionality or ruin the functionality of other interactive parts, so the feature was put aside while we worked on the live data part of the code.

> ☑ Blue VM 1 ☑ Blue VM 2

A second unimplemented feature is the VM filtering, which would allow users to only show data from one source or another, this feature would always break, so it wasn't fully implemented, but if it was removed it would also break everything else, so the buttons exist but don't do anything in the final version.